

REDES NEURONALES ARTIFICIALES

Desarrollo de los ejercicios propuestos en la sección 5.9

Ejercicio 1. Redes con dos capas ocultas para las compuertas NAND y XOR

Ejercicio 2. Clasificación de prendas de vestir con Fashion MNIST y TensorFlow

Ejercicio 3. Modelo con un conjunto de datos de Kaggle y ejemplos de internet

Ejercicio 4 (punto libre). De la convolución a la red convolucional

Presentado por:

- Edinson Sanchez Fuentes (edsanchezf@unal.edu.co)
- Brayan Alejandro Muñoz Pérez (bmunozp@unal.edu.co)
- Joan Sebastián Roberto Puerto (jroberto@unal.edu.co)

Docente: José J. Martínez P.
Universidad Nacional de Colombia, Sede Bogotá
Septiembre de 2026

Contenido

1. Introducción	3
1.1 Metodología	3
2. Ejercicio 1. Redes con dos capas ocultas para NAND y XOR.....	4
2.1 Planteamiento	4
2.2 Punto de partida: el programa de clase	4
2.3 El procedimiento del capítulo:	4
2.4 La derivada de la sigmoide en el programa de clase	6
2.5 Diseño de la red entrenada.....	6
2.6 Estructura del programa.....	7
2.7 Resultados de las redes entrenadas	8
2.8 Verificación y robustez.....	9
2.9 Conclusiones	9
3. Ejercicio 2. Clasificación de prendas con Fashion MNIST	11
3.1 Planteamiento y datos.....	11
3.2 Modelos.....	11
3.3 Estructura del programa y funciones principales	12
3.4 Resultados	13
3.5 Verificación.....	14
3.6 Conclusiones	14
4. Ejercicio 3. Modelo con un dataset de Kaggle.....	16
4.1 Elección del conjunto de datos	16
4.2 Características y rótulo	16
4.3 Calidad de los datos.....	17
4.4 Selección y entrenamiento del modelo	17
4.5 Ejemplos tomados de internet.....	18
4.6 Análisis de los resultados	19
4.7 Conclusiones	20
5. Ejercicio 4 (punto libre). De la convolución a la red convolucional	21
5.1 Planteamiento	21
5.2 Convolución y pooling desde cero.....	21
5.3 Red convolucional	22
5.4 Dentro de la red entrenada.....	22
5.5 Conclusiones	23
6. Observaciones sobre el material del capítulo	24
7. Uso de herramientas de IA Generativa	25
7.1 Herramientas empleadas.....	25
7.2 Prompts utilizados	25
7.3 Alcance y limitaciones observadas	25
8. Programas, repositorio y ejecución.....	27
8.1 Estructura de los archivos.....	27
8.2 Ejecución en Google COLAB	27
8.3 Ejecución local con Jupyter	27
9. Bibliografía	28

1. Introducción

Este documento presenta el desarrollo de los cuatro ejercicios propuestos en la sección 5.9 del capítulo *Introducción a las Redes Neuronales Artificiales*. Cada ejercicio se entrega como un notebook de Jupyter ejecutado, con sus resultados incluidos, y en este documento se describen sus partes, sus resultados y las conclusiones que se derivan de ellos, conforme a los parámetros de entrega del curso.

Los cuatro ejercicios recorren el capítulo en orden. El primero trabaja la neurona artificial, la propagación y la retropropagación directamente con NumPy, partiendo del programa de clase. El segundo usa TensorFlow para resolver un problema real de clasificación de imágenes. El tercero aplica una red a un conjunto de datos escogido por el grupo y la enfrenta a ejemplos nuevos tomados de internet. El cuarto, de tema libre, se dedicó a la sección 5.8, convolución y pooling.

1.1 Metodología

El principio que se siguió fue contrastar cada resultado con una referencia independiente en lugar de limitarse a presentarlo. Una red neuronal siempre produce una salida, y sin un punto de comparación no es posible distinguir un modelo que funciona de uno que solo produce números razonables. Las verificaciones realizadas fueron las siguientes:

- **Reproducción del material del capítulo.** Antes de construir sobre él, se reprodujeron las cifras exactas del Ejemplo 1 (sección 5.6), la matriz 4×4 de la convolución y el max pooling de la figura 5.23 (sección 5.8), y la figura 5.24.
- **Comprobación de las derivadas.** El gradiente calculado por la retropropagación se comparó con derivadas numéricas por diferencias centrales.
- **Repetición de las operaciones de TensorFlow.** Con los pesos extraídos de las redes entrenadas se repitió la propagación con NumPy, para confirmar que la librería calcula exactamente las ecuaciones del capítulo.
- **Líneas base.** Cada red se comparó con un modelo más simple: una neurona sin capas ocultas, una red sin regularización.
- **Robustez.** Cuando el resultado podía depender de la inicialización, el entrenamiento se repitió con varias semillas aleatorias.

2. Ejercicio 1. Redes con dos capas ocultas para NAND y XOR

Enunciado: con base en el programa Clase_NNA3, desarrolle dos redes neuronales con 2 capas ocultas, una que aprenda a reconocer una compuerta NAND y la otra aprenda a reconocer una compuerta XOR.

2.1 Planteamiento

Las dos compuertas tienen dos entradas binarias y una salida binaria. La NAND es linealmente separable: una sola recta divide el punto (1,1) de los otros tres. La XOR no lo es: los dos puntos con salida 1, (0,1) y (1,0), están en esquinas opuestas del cuadrado. Es la limitación que Minsky y Papert señalaron en *Perceptrones* y que, según el capítulo, frenó el área durante cerca de diez años. Resolverla requiere capas ocultas, y eso es lo que el ejercicio pone a prueba.

Tabla 1. Tabla de verdad de las dos compuertas.

x1	x2	NAND	XOR
0	0	1	0
0	1	1	1
1	0	1	1
1	1	0	0

El desarrollo se organizó en dos partes que conviene distinguir desde el principio. En la primera se reproduce el procedimiento de la sección 5.3 del capítulo, donde los pesos se escogen a mano y las compuertas se encadenan como en un circuito lógico. En la segunda se entrenan las dos redes de dos capas ocultas que pide el enunciado, partiendo de pesos aleatorios, que es lo que significa que aprenda a reconocer.

2.2 Punto de partida: el programa de clase

Se transcribió el programa del Ejemplo 1 (red 2-2-2 con los pesos de la figura, entrada [0.05, 0.10] y rótulos [0.01, 0.99]). Sus salidas coinciden con las impresas en el capítulo hasta el octavo decimal: [0.69051105, 0.72421384] antes de entrenar, [0.67821878, 0.72890846] tras una actualización y [0.03341429, 0.9900029] tras 50 000 épocas. Esto confirma que se parte del mismo código.

2.3 El procedimiento del capítulo:

La sección 5.3 del capítulo construye varias compuertas escogiendo los pesos a mano, sin entrenamiento. Reproducir ese procedimiento fija la notación que se usa después (W^1 , Z^1 , A^1 , W^2 , Z^2 , $g(Z)$), muestra de dónde sale la NAND de dos capas y da una referencia contra la cual comparar lo que la red aprenda por su cuenta. En toda esta parte la activación es la sigmoide y los pesos son grandes (± 10 , ± 20 , ± 30) para llevarla a saturación, de modo que las salidas queden muy cerca de 0 o de 1.

Compuertas de una sola neurona. Las figuras 5.7, 5.8 y 5.9 resuelven tres compuertas cambiando únicamente el peso del sesgo, lo que ilustra la afirmación del capítulo de que *el cambio de peso en el vector de sesgo cambia la función de la red neuronal*. Las tres se verificaron y dan su tabla de verdad.

Tabla 2. Compuertas de una neurona del capítulo, verificadas en el notebook.

Compuerta	Figura	Expresión de Z	Salida obtenida
AND	5.7	$Z = -30 + 20x_1 + 20x_2$	(0, 0, 0, 1) correcta
OR	5.8	$Z = -10 + 20x_1 + 20x_2$	(0, 1, 1, 1) correcta
NOT	5.9	$Z = 10 - 20x_1$	(1, 0) correcta

NAND con dos capas (figura 5.10). Este es el procedimiento que interesa al ejercicio. El capítulo encadena las dos redes anteriores: la primera capa calcula una AND con el vector de pesos (b, w_1, w_2) = (-30, 20, 20), y la segunda aplica una NOT con (b_2, W^2) = (10, -20). En palabras del capítulo, $Z^1 = (-30 + 20x_1 + 20x_2)$, luego $A^1 = g(Z^1)$; luego $Z^2 = (10a_1 - 20a_2)$ y $A^2 = g(Z^2)$. El notebook reproduce la tabla de la figura 5.10 con sus mismas once columnas.

Tabla 3. Tabla de clasificación de la compuerta NAND, reproducida de la figura 5.10 con los pesos y el cálculo del capítulo.

X1	X2	b	w1	w2	Z ¹	g(Z ¹)	b2	W ²	Z ²	g(Z ²)
0	0	-30	20	20	-30	~0	10	-20	10	~1
0	1	-30	20	20	-10	~0	10	-20	-10	~0
1	0	-30	20	20	-10	~0	10	-20	-10	~0
1	1	-30	20	20	10	~1	10	-20	-30	~0

El notebook obtiene estos mismos valores en las once columnas. Conviene dejar anotado un dato numérico para quien lea el trabajo: la columna $g(Z^2)$ de esta tabla es (1, 0, 0, 0), mientras que la tabla de verdad de la NAND, que es la que se usa como rótulo al entrenar la red en la sección 2.7 y la que aparece en la Tabla 1, es (1, 1, 1, 0).

NOR y la red de dos salidas (figura 5.11). El capítulo continúa indicando que la NOR se obtiene de la misma forma, uniendo las conformaciones OR y NOT: basta cambiar el sesgo de la primera capa de -30 a -10. Después reúne ambas construcciones en una sola red con dos neuronas ocultas y dos salidas, donde la capa 1 calcula la AND y la OR en paralelo y la capa 2 niega cada una, de modo que $y_1 = \text{NAND}$ y $y_2 = \text{NOR}$ simultáneamente. Es el caso de multi-clasificación con $k = 2$ que menciona el texto. La red se verificó y entrega las dos compuertas correctas.

XOR con las mismas piezas. Puede armarse con las compuertas anteriores mediante la identidad $\text{XOR} = \text{AND}(\text{OR}, \text{NAND})$. La capa oculta contiene la OR del capítulo (-10, 20, 20) y una NAND expresada en una neurona (30, -20, -20), y la salida es la AND del capítulo (-30, 20, 20). Se verificó que produce (0, 1, 1, 0). Esto deja establecido un punto importante: la XOR se resuelve

con una sola capa oculta, de modo que las dos capas que pide el enunciado dan más capacidad de la estrictamente necesaria.

2.4 La derivada de la sigmoide en el programa de clase

Al revisar el código del Ejemplo 1 se encontró que la función dSig calcula $\text{sig}(s.T[0]) * (\text{sig}(s.T[0]))$, es decir σ^2 , y no la derivada $\sigma(1 - \sigma)$ que el propio capítulo establece en las secciones 5.2.2 y 5.5. La comparación con la derivada numérica lo confirma (Figura 1). El error pasa inadvertido porque σ^2 también es positiva y, en la capa de salida, el gradiente sigue apuntando en una dirección de descenso; pero su escala es incorrecta, sobre todo cuando la neurona está saturada. Con la derivada corregida, el Ejemplo 1 llega a [0.0116, 0.9885] y su error final se reduce 110 veces (de 2.7×10^{-4} a 2.5×10^{-6}). La primera salida, que el capítulo señala como pendiente, alcanza su rótulo de 0.01.

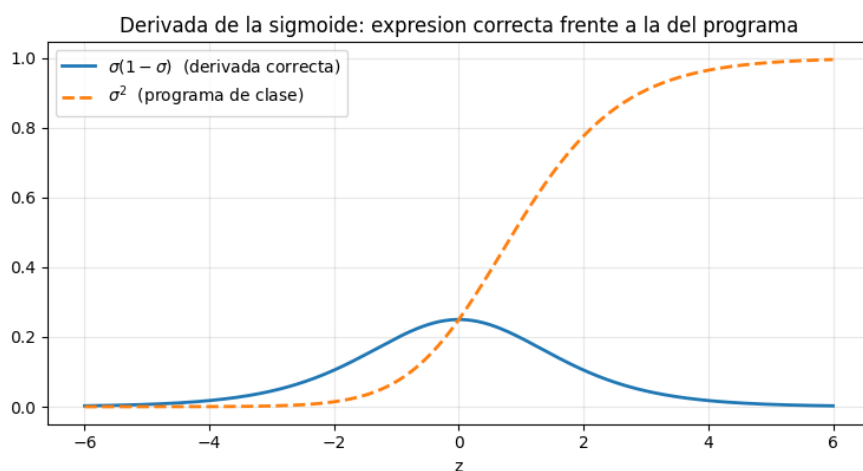


Figura 1. Derivada correcta de la sigmoide, $\sigma(1 - \sigma)$, frente a la expresión σ^2 que usa el programa de clase.

2.5 Diseño de la red entrenada

Para la parte que pide el enunciado se usó la topología 2-3-3-1: dos entradas, dos capas ocultas de tres neuronas y una neurona de salida, con activación sigmoide en todas las capas. Son 25 parámetros ($9 + 12 + 4$). Dos neuronas ocultas bastarían, como mostró la construcción manual de la XOR; la tercera da holgura y hace el entrenamiento menos propenso a estancarse.

Las ecuaciones son las de la sección 5.5 del capítulo, que generalizan las de la sección 2.3 a más capas. Para $k = 1, 2, 3$:

$$\begin{aligned} Z^{(k)} &= W^{(k)} A^{(k-1)} + B^{(k)}, & A^{(k)} &= \sigma(Z^{(k)}) \\ \delta^{(3)} &= \text{diag}(\sigma'(Z^{(3)})) e, & \delta^{(k)} &= \text{diag}(\sigma'(Z^{(k)})) W^{(k+1)T} \delta^{(k+1)} \\ \partial E / \partial W^{(k)} &= -\delta^{(k)} A^{(k-1)T}, & \partial E / \partial B^{(k)} &= -\delta^{(k)}, & W &\leftarrow W - \eta \partial E / \partial W \end{aligned}$$

Respecto al programa de clase se hicieron dos cambios. Primero, se entrena con los cuatro patrones a la vez: X es una matriz 2×4 cuyas columnas son los patrones, y el producto $\delta A^{(k-1)T}$ suma automáticamente sus contribuciones. Segundo, la matriz diagonal se reemplaza por un producto elemento a elemento, que es equivalente y funciona con varias columnas. Se conservaron los

nombres de las funciones del programa de clase. Como comprobación de que la implementación general es correcta, se le cargaron los pesos de la figura 5.10 y reprodujo exactamente la tabla de la sección 2.3.

Tabla 4. Hiperparámetros de las redes entrenadas.

Parámetro	Valor	Comentario
Topología	2-3-3-1	dos capas ocultas de tres neuronas
Activación	sigmoide	con la derivada corregida
Inicialización	uniforme en $[-1, 1]$	se compara con $[0, 1]$ en la sección 2.8
Tasa de aprendizaje η	0.5	la pérdida se suma sobre 4 patrones
Épocas	20 000	cada época presenta los cuatro patrones
Semillas	1 (principal); 100 a 119 (robustez)	resultados reproducibles

2.6 Estructura del programa

Tabla 5. Secciones del notebook del Ejercicio 1.

Sección	Contenido
1. Planteamiento	Tablas de verdad, separabilidad lineal y plan del notebook.
2. Programa de clase	Reproducción del Ejemplo 1.
3. Compuertas a mano	Procedimiento de la sección 5.3: AND, OR y NOT de una neurona; NAND de la figura 5.10 reproducida con sus once columnas; NOR y la red de dos salidas de la figura 5.11; XOR armada con las mismas piezas.
4. Derivada de la sigmoide	Comparación con la derivada numérica y efecto de la corrección en el Ejemplo 1.
5. Generalización	Funciones sig, dSig, inicializa, propaga, error, backpropagation y entrena, y comprobación contra la figura 5.10.
6. Verificación del gradiente	Comparación de los 25 gradientes con diferencias centrales.
7. Red NAND entrenada	Entrenamiento, tabla de salidas, frontera de decisión y comparación con la red construida a mano.
8. Red XOR entrenada	Entrenamiento, frontera de decisión y activaciones internas.
9. Una sola neurona	Comprobación de la limitación de Minsky y Papert.
10. Robustez	160 entrenamientos: 2 compuertas $\times$ 2 derivadas $\times$ 2 inicializaciones $\times$ 20 semillas.
11. Conclusiones	Síntesis de las dos partes y de los hallazgos sobre el material del capítulo.

2.7 Resultados de las redes entrenadas

Las dos redes aprenden sus compuertas con salidas a menos de 0.01 de los r tulos. En la NAND la salida m s alejada es 0.0077 para el patr n (1,1); en la XOR, 0.9931 para (1,0). La p rdida final es del orden de 8×10^{-5} en ambos casos. La red NAND entrenada llega a salidas tan pr ximas a 0 y 1 como la construida a mano en la secci n 2.3, pero con pesos que encontr  el algoritmo.

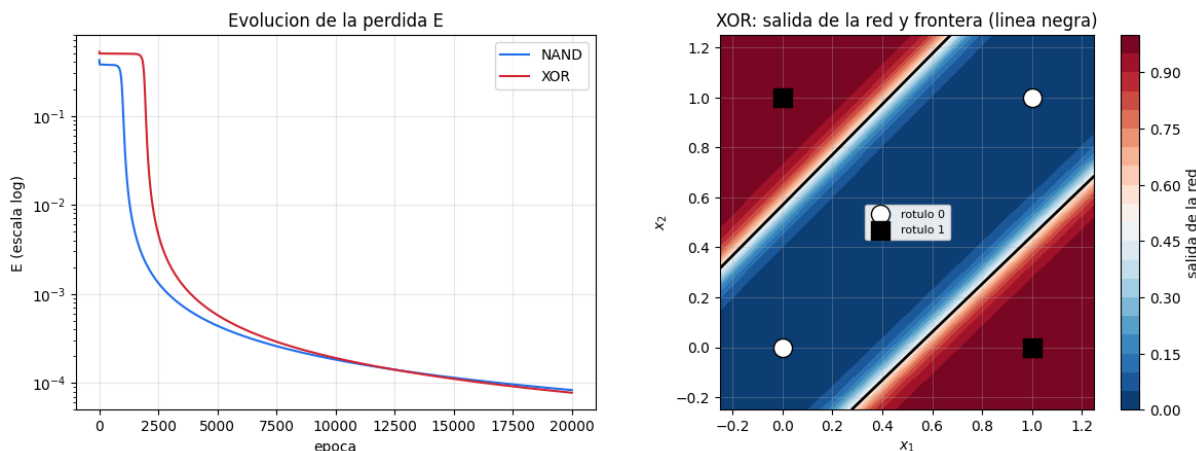


Figura 2. Izquierda: evoluci n de la p rdida de las dos redes. Derecha: salida de la red XOR en todo el plano y su frontera de decisi n.

La curva de la XOR tiene una meseta de cerca de 2 000  pocas en $E \approx 0.5$, que corresponde a responder 0.5 a los cuatro patrones. Es el periodo en que la red todav a no ha encontrado c mo reorganizar los datos en las capas ocultas; cuando lo hace, la p rdida cae cuatro  rdenes de magnitud. La frontera de decisi n de la XOR est  formada por dos rectas paralelas, una forma que ninguna neurona individual puede producir.

La Figura 3 muestra c mo lo consigue la red. En la entrada, los dos patrones con salida 1 est n en esquinas opuestas. En la segunda capa oculta, los patrones de cada clase pr cticamente coinciden en un mismo punto (00 con 11 y 01 con 10), y la neurona de salida, que solo puede trazar un plano, ya puede separarlos. Es la idea central del cap tulo: las capas ocultas con activaci n no lineal cambian la representaci n de los datos. La red entrenada llega al mismo resultado que la construcci n manual de la secci n 2.3, pero por un camino distinto: all  las dos neuronas ocultas eran una OR y una NAND reconocibles, mientras que aqu  las tres neuronas de cada capa reparten el trabajo sin corresponder a compuertas con nombre.

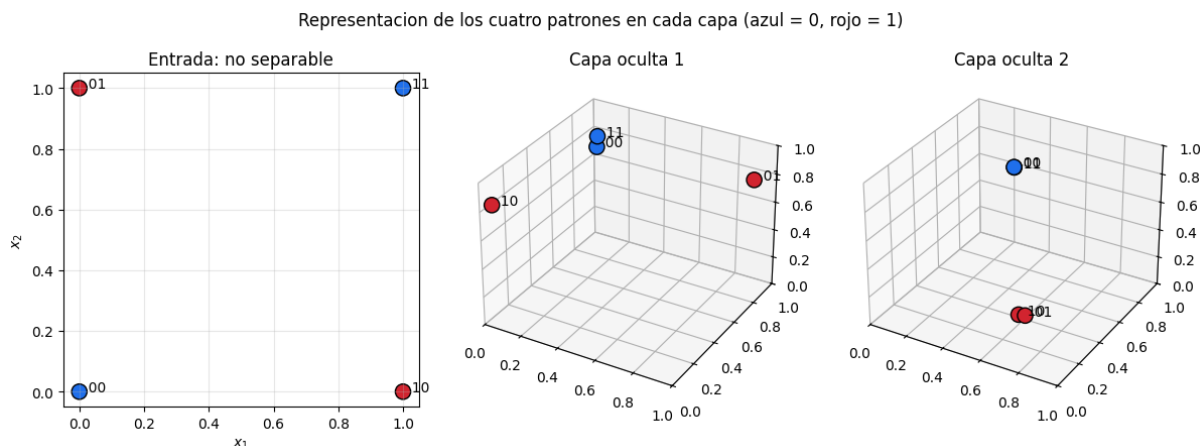


Figura 3. Representación de los cuatro patrones de la XOR en la entrada y en cada capa oculta (azul: rótulo 0; rojo: rótulo 1).

Una sola neurona sin capas ocultas, entrenada con el mismo algoritmo, aprende la NAND pero en la XOR se estanca respondiendo 0.5 a los cuatro patrones, lo que reproduce la limitación señalada por Minsky y Papert.

2.8 Verificación y robustez

Gradiente. Con la derivada corregida, los 25 gradientes calculados por retropropagación coinciden con los numéricos con un error relativo máximo de 2.3×10^{-7} . Con la derivada del programa de clase el error relativo es 1.0: lo que se calcula no es el gradiente de la pérdida.

Tabla 6. Entrenamientos exitosos en 20 semillas (todas las salidas a menos de 0.1 de su rótulo).

Compuerta	Inicialización	Derivada $\sigma(1-\sigma)$	Derivada σ^2 (capítulo)
NAND	[0, 1]	20/20	6/20
NAND	[-1, 1]	20/20	17/20
XOR	[0, 1]	20/20	0/20
XOR	[-1, 1]	19/20	0/20

Con la derivada correcta, 79 de 80 entrenamientos llegan a la solución. El único fallo (XOR, semilla 113) es un mínimo local clásico del problema: la red resuelve dos patrones y responde 0.5 a los otros dos, con $E = 0.25$. Con la derivada del capítulo la XOR no se aprende en ninguna de las 40 corridas. El error del programa pasa inadvertido en el Ejemplo 1, donde basta con acercar dos salidas a sus rótulos, pero impide el aprendizaje cuando las capas ocultas tienen que construir una representación nueva.

2.9 Conclusiones

1. Se reprodujo el procedimiento de la sección 5.3 con el que el capítulo construye las compuertas escogiendo los pesos a mano: AND, OR y NOT con una sola neurona; la red de dos capas de la figura 5.10, con sus once columnas y los mismos pesos (-30, 20, 20) y (10,

–20); la NOR obtenida de la misma forma a partir de la OR; la red de dos salidas de la figura 5.11; y la XOR armada con esas mismas piezas, lo que muestra que basta una capa oculta para resolverla.

2. Las dos redes que pide el enunciado, con topología 2-3-3-1 y entrenadas por retropropagación desde pesos aleatorios, aprenden sus compuertas de forma confiable: 79 de 80 corridas con 20 semillas distintas. La XOR, imposible para una sola neurona, se resuelve porque las capas ocultas reubican los patrones hasta hacerlos separables por un plano.
3. Reproducir las cifras exactas del Ejemplo 1 permitió detectar que dSig calcula σ^2 en lugar de $\sigma(1 - \sigma)$. Con la corrección, el propio ejemplo del capítulo mejora 110 veces su error, y la XOR pasa de no aprenderse nunca a aprenderse en 39 de 40 corridas.
4. El contraste entre construir y aprender es lo que da sentido al ejercicio. Con pesos escogidos a mano, cada neurona tiene un significado claro y las compuertas se encadenan como en un circuito lógico. Con pesos aprendidos, la red llega a la misma función sin que ninguna neurona corresponda a una compuerta reconocible: el conocimiento queda repartido entre los 25 parámetros.

3. Ejercicio 2. Clasificación de prendas con Fashion MNIST

Enunciado: con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Explique cada una de las funciones y las principales instrucciones, saque conclusiones.

3.1 Planteamiento y datos

Fashion MNIST (Zalando Research) conserva el formato del MNIST de dígitos descrito en la sección 5.4: imágenes en escala de grises de $28 \times 28 = 784$ píxeles, 60 000 de entrenamiento, 10 000 de prueba y 10 clases balanceadas (6 000 ejemplos cada una). Cada imagen es una prenda o un accesorio: camiseta, pantalón, buzo, vestido, abrigo, sandalia, camisa, tenis, bolso y botín. Es más difícil que MNIST porque varias clases se parecen mucho entre sí.



Figura 4. Primeras 24 imágenes del conjunto de entrenamiento con su rótulo.

Los píxeles se normalizaron a $[0, 1]$ dividiéndolos por 255. El conjunto se dividió, siguiendo la sección 5.4 del capítulo, en 50 000 ejemplos de entrenamiento, 10 000 de validación (para vigilar el sobreentrenamiento) y 10 000 de prueba, que se usaron una sola vez al final.

3.2 Modelos

Tabla 7. Modelos entrenados en el Ejercicio 2 y su exactitud en el conjunto de prueba.

Modelo	Arquitectura	Parámetros	Épocas	Pérdida	Exactitud
Referencia	784 $\rightarrow$ 10 softmax, sin capas ocultas	7 850	10	0.456	84.0 %
Red con capa oculta	784 $\rightarrow$ 128 ReLU $\rightarrow$ 10 softmax	101 770	30	0.538	86.7 %
Red regularizada	igual, con Dropout(0.2) y parada temprana	101 770	17 (mejor: 12)	0.342	88.3 %

El modelo de referencia es el equivalente de las compuertas de una sola neurona, con diez salidas. La red con capa oculta usa ReLU (figura 5.7 del capítulo) y una salida softmax, que convierte las diez salidas en probabilidades que suman 1. La tercera añade dos técnicas contra el sobreentrenamiento: *Dropout*, que apaga al azar el 20 % de las neuronas ocultas en cada lote durante el entrenamiento, y la parada temprana, que detiene el entrenamiento cuando la pérdida de validación deja de mejorar y restaura los mejores pesos.

3.3 Estructura del programa y funciones principales

El notebook explica cada instrucción inmediatamente después de usarla. La Tabla 8 resume las funciones de TensorFlow utilizadas y su relación con los conceptos del capítulo.

Tabla 8. Funciones principales de TensorFlow utilizadas.

Función	Qué hace	Relación con el capítulo
<code>fashion_mnist.load_data()</code>	Descarga el conjunto y lo entrega como arreglos de NumPy	pares (X, Y) de entrenamiento
<code>keras.Sequential</code>	Red <i>feedforward</i> organizada por capas	figura 5.13, propagación
<code>layers.Flatten</code>	Convierte la matriz 28×28 en un vector de 784	vector de características X
<code>layers.Dense(n, activation)</code>	Capa completamente conectada: $A = g(W^T A + B)$	neurona artificial, sección 5.2.2
<code>relu / softmax</code>	Activaciones oculta y de salida	figura 5.7; multi-clasificación con $k = 10$
<code>layers.Dropout(p)</code>	Apaga neuronas al azar durante el entrenamiento	regularización
<code>model.compile</code>	Asigna optimizador, pérdida y métricas	algoritmo de aprendizaje, sección 5.5
<code>optimizers.Adam</code>	Descenso del gradiente con tasa adaptativa	$W \leftarrow W - \eta \nabla E$
<code>SparseCategoricalCrossentropy</code>	Pérdida $E = -\ln p$ de la clase correcta	función de pérdida
<code>model.fit</code>	Épocas y lotes de propagación y retropropagación	<i>Backpropagation</i>
<code>callbacks.EarlyStopping</code>	Detiene al dejar de mejorar la validación	figuras 5.12 y 5.17
<code>model.evaluate / predict</code>	Pérdida y exactitud; probabilidades de salida	estimación con prueba; $H_w(X)$
<code>model.get_weights</code>	Devuelve las matrices W y vectores B	verificación manual
<code>tf.math.confusion_matrix</code>	Tabla de clase real por clase predicha	análisis de errores

Se destacan dos decisiones. La pérdida de entropía cruzada reemplaza al error cuadrático del capítulo porque, combinada con softmax, produce un gradiente que no se satura y castiga las respuestas seguras y equivocadas. El optimizador Adam es una variante del descenso del gradiente que adapta la tasa de aprendizaje de cada peso; los gradientes los calcula TensorFlow por retropropagación automática.

3.4 Resultados

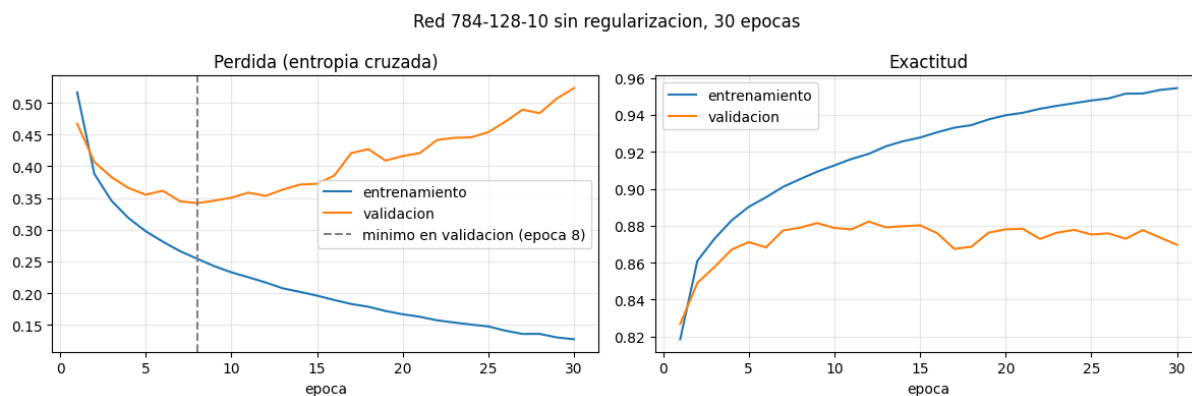


Figura 5. Red sin regularizar: pérdida y exactitud de entrenamiento y validación durante 30 épocas.

La red sin regularizar reproduce el comportamiento de la figura 5.17 del capítulo: la pérdida de entrenamiento baja durante las 30 épocas, pero la de validación alcanza su mínimo hacia la época 8 y luego sube. Al final, la exactitud es 95.5 % en entrenamiento y solo 87.0 % en validación. En prueba, su pérdida (0.54) es incluso peor que la del modelo sin capas ocultas (0.46): la red sobreentrenada se equivoca con mucha seguridad. La red regularizada corrige esto y obtiene el mejor resultado, 88.3 % de exactitud con una pérdida de 0.34.

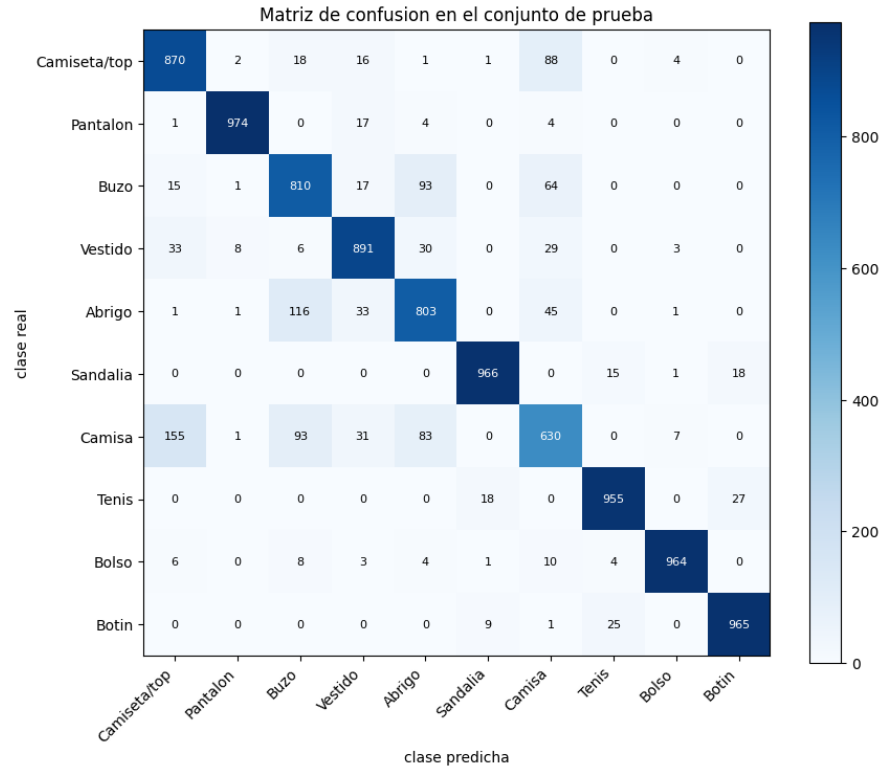


Figura 6. Matriz de confusión de la red regularizada en el conjunto de prueba.

Los errores no están repartidos al azar. Pantalones, bolsos, sandalias, tenis y botines se reconocen por encima del 95 %. La clase más difícil es Camisa, con 63 % de aciertos, que se confunde sobre todo con camiseta (155 casos), buzo y abrigo. Las prendas superiores con mangas tienen una silueta casi idéntica a 28×28 píxeles, y al revisar los errores se encuentran imágenes ambiguas incluso para una persona.

3.5 Verificación

Con los pesos extraídos mediante `get_weights()` se repitió la propagación con NumPy: $Z^1 = XW^1 + B^1$, ReLU, $Z^2 = A^1W^2 + B^2$ y softmax. Las probabilidades coinciden con las de Keras con una diferencia máxima del orden de 10^{-6} , atribuible a la precisión simple, y las 10 000 predicciones de clase son idénticas. TensorFlow calcula exactamente las ecuaciones del capítulo; lo que aporta es la retropropagación automática y la eficiencia. El notebook también verifica a mano el número de parámetros: $784 \cdot 128 + 128 = 100\,480$ y $128 \cdot 10 + 10 = 1\,290$.

3.6 Conclusiones

1. Una red con una capa oculta de 128 neuronas clasifica correctamente el 88.3 % de las prendas de prueba, frente a 84.0 % sin capas ocultas y 10 % al azar.
2. El sobreentrenamiento aparece hacia la época 8 y se reconoce en la divergencia de las curvas de entrenamiento y validación. *Dropout* y la parada temprana lo controlan de forma automática.

3. Keras implementa exactamente las ecuaciones del capítulo, como lo demuestra la propagación repetida a mano.
4. La mayoría de los errores ocurre entre prendas superiores de silueta parecida. Una red densa trata cada píxel como una característica independiente y no aprovecha la estructura espacial de la imagen, limitación que aborda el Ejercicio 4.

4. Ejercicio 3. Modelo con un dataset de Kaggle

Enunciado: de la librería de Kaggle para Python, baje un dataset, estudie sus características (features) y su rótulo. Genere un modelo y tome ejemplos de internet y analice los resultados.

4.1 Elección del conjunto de datos

Se usó Pokemon with stats (abcsds/pokemon), que describe los 800 Pokémon de las seis primeras generaciones de la franquicia, incluidas sus formas alternativas, con seis estadísticas de combate y un rótulo binario que indica si el Pokémon es legendario. Se descargó con kagglehub, la librería oficial de Kaggle para Python. Se eligió por tres razones:

- **Rótulo y características claros:** seis estadísticas numéricas y un rótulo que corresponde a una categoría oficial del juego.
- **Ejemplos nuevos verificables en internet:** entre 2016 y 2022 se publicaron las generaciones 7, 8 y 9, que el conjunto no contiene. Sus estadísticas y su categoría están publicadas en PokémonDB y Bulbapedia, de modo que se puede probar el modelo con ejemplos que nunca vio y cuya respuesta se conoce con certeza.
- **Problemas reales de aprendizaje de máquina:** fuerte desbalance de clases (8 % de legendarios), problemas de calidad de datos y un límite conceptual: la condición de legendario es una decisión narrativa de los creadores del juego, no una propiedad física.

En el entorno donde se ejecutó el notebook no había acceso a la API de Kaggle, así que se usó el respaldo previsto en el código: una copia idéntica del archivo alojada en GitHub. Los valores de control (800 filas, 65 legendarios y la columna Total igual a la suma de las estadísticas) coinciden con la ficha del conjunto en Kaggle. En Colab, kagglehub descarga el conjunto directamente.

4.2 Características y rótulo

Tabla 9. Columnas del conjunto y decisión tomada para cada una.

Columna	Descripción	Decisión
#, Name	Número de Pokédex y nombre	Excluidas: identificadores
Type 1, Type 2	Tipo elemental (18 categorías)	Evalúadas en la selección de modelo; empeoraron el resultado
Total	Suma de las seis estadísticas	Excluida de la red por redundante; usada en la regla base
HP, Attack, Defense, Sp. Atk, Sp. Def, Speed	Estadísticas de combate	Características (n = 6)
Generation	Generación 1 a 6	Excluida: los ejemplos nuevos son de las generaciones 7 a 9
Legendary	Legendario sí/no	Rótulo

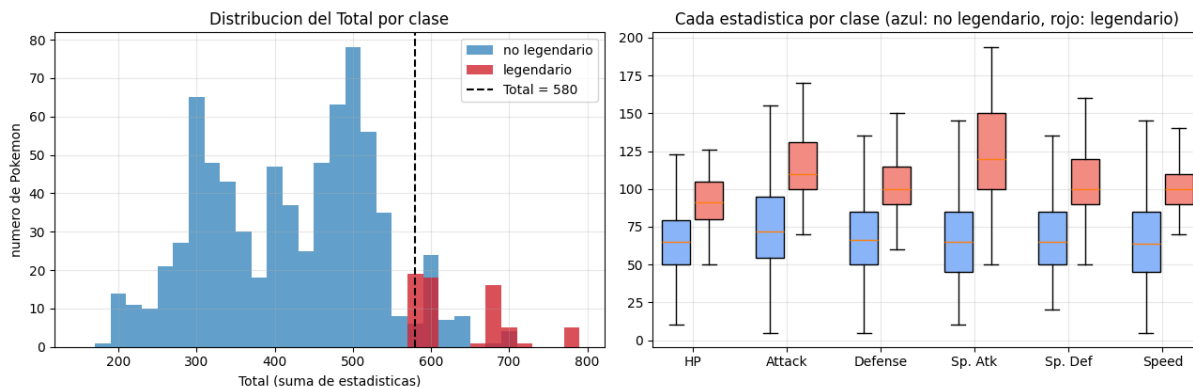


Figura 7. Distribución del Total y de cada estadística por clase en el conjunto de Kaggle.

La exploración muestra que todos los legendarios del conjunto tienen un Total de al menos 580. Eso sugiere una regla trivial, legendario si $\text{Total} \geq 580$, que se usó como línea base: la red solo se justifica si mejora una regla que cualquiera escribiría mirando el histograma.

4.3 Calidad de los datos

Formas alternativas. El conjunto incluye como filas separadas 50 megaevoluciones y regresiones primigenias, que son transformaciones temporales de un Pokémon ya presente. Casi todos los no legendarios con Total alto son megaevoluciones de Pokémon comunes (31 de 48), con estadísticas infladas artificialmente. Se eliminaron: duplican especies y contradicen el patrón a aprender.

Rótulos inconsistentes. Al revisar los errores de un primer modelo aparecieron Celebi y Keldeo, clasificados como legendarios por la red pero rotulados como comunes. Ambos son míticos, una categoría oficial que el conjunto trata de forma inconsistente: Jirachi, Darkrai o Arceus, también míticos, sí figuran como legendarios. Se contrastó la columna con las listas oficiales de legendarios y míticos de Bulbapedia y se adoptó la definición "legendario o mítico", que es la que usa el conjunto en la mayoría de los casos. Diez filas de ocho especies estaban mal rotuladas (Mew, Celebi, Phone, Manaphy, Keldeo, Meloetta, Genesect y Cresselia), todas como comunes. El conjunto final tiene 750 Pokémon, de los cuales 67 son legendarios o míticos.

4.4 Selección y entrenamiento del modelo

Se usó una red 6-16-8-1, con dos capas ocultas ReLU, una salida sigmoide y 257 parámetros, entrenada con entropía cruzada binaria, Adam ($\eta = 0.01$), características estandarizadas y pesos de clase para compensar el desbalance. Como hay pocos legendarios, las variantes se compararon con validación cruzada estratificada de 5 particiones, usando métricas que se fijan en la clase minoritaria (con 92 % de comunes, responder siempre "no legendario" da 92 % de exactitud).

Tabla 10. Validación cruzada de 5 particiones: F1 promedio de la red y de la regla $\text{Total} \geq 580$.

Variante	F1 red	AUC red	F1 regla
A. Datos originales de Kaggle	0.646	0.963	0.734
B. A sin formas alternativas	0.811	0.979	0.870

Variante	F1 red	AUC red	F1 regla
C. B con rótulo corregido	0.799	0.976	0.937
D. C + 18 variables de tipo	0.728	0.955	0.937

Quitar las formas alternativas es la decisión que más mejora la red: su F1 pasa de 0.65 a 0.81. Corregir el rótulo favorece sobre todo a la regla, porque los casos corregidos tienen Totales de 580 a 600; se mantiene de todos modos, porque son datos equivocados. Agregar los tipos empeora la red por sobreentrenamiento. Y hay un resultado que conviene decir sin rodeos: con los datos de Kaggle, la regla supera a la red en F1 (0.94 frente a 0.80). La red ordena muy bien a los Pokémon (AUC de 0.98), pero produce más falsas alarmas.

El modelo final (variante C) se entrenó con una división estratificada 70/15/15 y parada temprana. En prueba detecta los 10 legendarios, pero marca además a 6 comunes (F1 de 0.77), mientras que la regla obtiene un F1 de 0.95. Se ajustó el umbral de decisión con el conjunto de validación, que sugirió 0.4, sin cambios en prueba: las falsas alarmas no se deben a un umbral mal puesto sino a lo que la red aprendió.

4.5 Ejemplos tomados de internet

Se tomaron de PokémonDB, y se contrastaron con Bulbapedia, las estadísticas oficiales de 23 Pokémon de las generaciones 7, 8 y 9, elegidos para cubrir las categorías que ponen a prueba el modelo.

Tabla 11. Resultados con los ejemplos de internet por categoría (aciertos de cada modelo).

Categoría	Ejemplos	Rótulo	Red	Regla
Legendarios de portada	Solgaleo, Zacian, Eternatus, Koraidon, Miraidon	1	5/5	5/5
Mítico	Zeraora	1	1/1	1/1
Legendarios menores	Tapu Koko, Ting-Lu, Calyrex	1	1/3	0/3
Legendarios en etapa inicial	Kubfu, Cosmog	1	0/2	0/2
Pseudolegendarios	Dragapult, Baxcalibur, Kommo-o	0	0/3	0/3
Ultraente y Paradoja	Buzzwole, Great Tusk	0	2/2	2/2
Comunes	Kingambit, Gholdengo, Decidueye, Corviknight, Toxapex, Mimikyu, Wooloo	0	6/7	7/7
Total			15/23	15/23

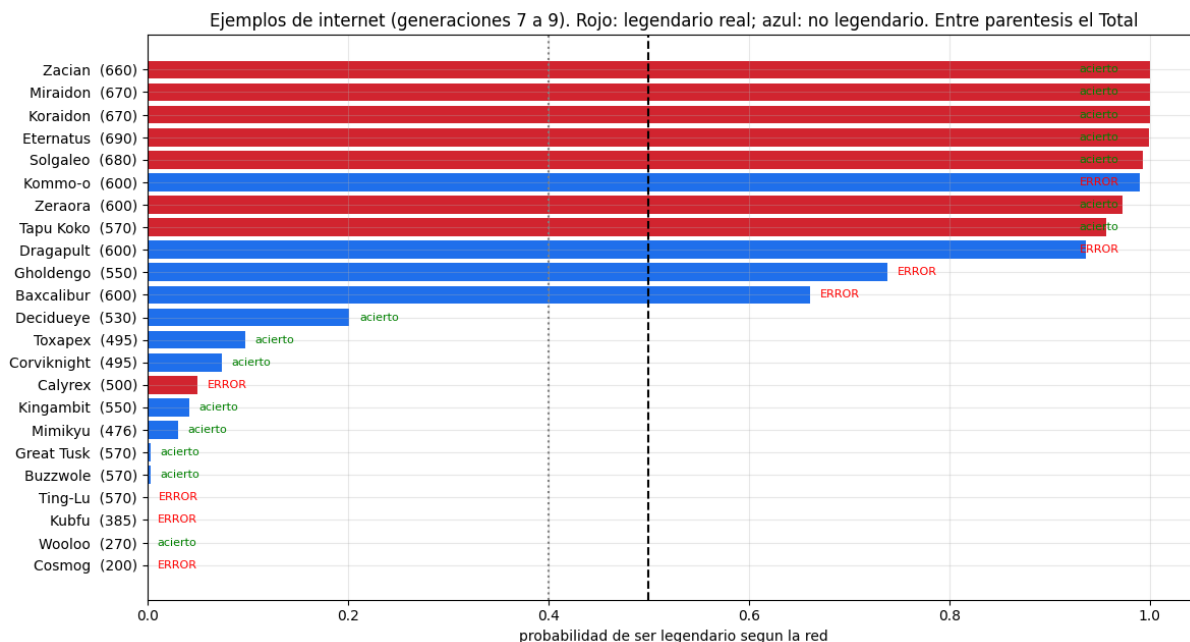


Figura 8. Probabilidad de ser legendario asignada por la red a cada ejemplo de internet (rojo: legendario real; azul: no legendario).

4.6 Análisis de los resultados

Los errores no son aleatorios y cada grupo muestra algo distinto. Ningún modelo reconoce a Cosmog, Kubfu y Calyrex, legendarios con Totales de 200 a 500: su condición viene de la historia del juego, no de sus estadísticas, y en los datos de entrenamiento solo había un legendario con Total bajo (Phione). Los pseudolegendarios, con Total de 600, engañan a ambos modelos porque con estas seis variables son indistinguibles de un legendario.

La diferencia entre los modelos aparece en los casos intermedios. Para entender qué aprendió la red se compararon las medianas de los Pokémon fuertes ($Total \geq 550$) del conjunto de Kaggle: los no legendarios tienen mucho más *Attack* (132 frente a 101.5) y menos *Sp. Atk* (105 frente a 117.5) que los legendarios. Los comunes más fuertes de las primeras generaciones son sobre todo atacantes físicos (Dragonite, Tyranitar, Garchomp, Salamence, Metagross). La red aprendió esa asociación por sí sola, como lo muestra el análisis de sensibilidad (Figura 9): si el *Sp. Atk* de Buzzwole subiera de 53 a 110 sin tocar nada más, su probabilidad pasaría de 0.003 a 0.59, mientras que variar su *Attack* no la cambia.

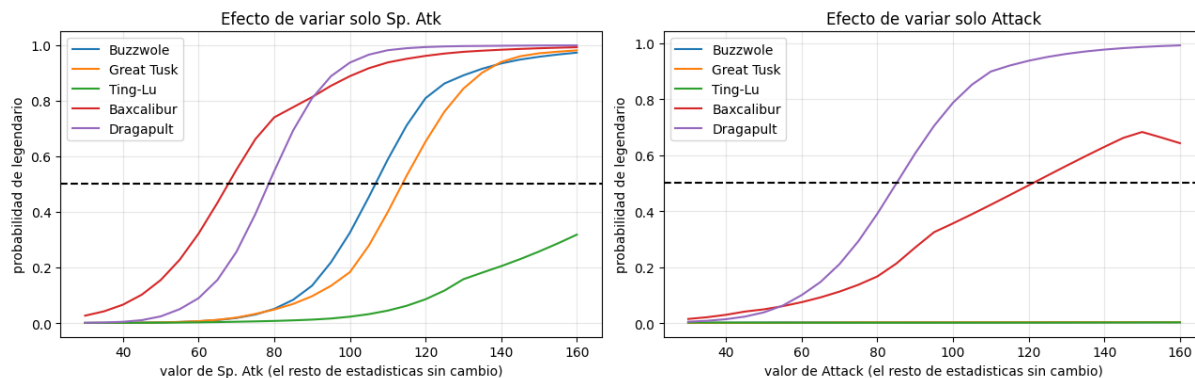


Figura 9. Probabilidad asignada por la red al variar solo el ataque especial (izquierda) o solo el ataque físico (derecha) de cinco ejemplos.

Ese patrón explica tanto los aciertos como los errores. La red rechaza bien a Buzzwole, Great Tusk y Kingambit, atacantes físicos con *Sp. Atk* de 53 a 60, y acepta a Tapu Koko (570), que la regla descarta. Pero falla con Ting-Lu, un legendario que también es atacante físico, y con Gholdengo, un común con *Sp. Atk* de 133. Una comprobación de rango mostró que los ejemplos nuevos no están fuera de lo que la red conoció (solo la velocidad de Ting-Lu queda por debajo del mínimo observado): los errores no vienen de extrapolar, sino de que el patrón aprendido no siempre se cumple en las generaciones nuevas.

4.7 Conclusiones

1. Estudiar las características y el rótulo fue la parte más productiva del ejercicio. Se detectaron 50 filas de formas alternativas y 10 rótulos equivocados. Eliminar las formas mejoró el F1 de la red más que cualquier cambio de arquitectura.
2. Revisar los errores del modelo sirvió para auditar los datos: la red tenía razón con Celebi y Keldeo, y el rótulo estaba mal.
3. Una red neuronal no es automáticamente mejor que una regla simple. En los datos de Kaggle la regla $\text{Total} \geq 580$ obtiene mejor F1, y compararse con ella fue indispensable para interpretar los resultados.
4. Con los ejemplos de internet, red y regla empatan en aciertos (15 de 23) pero fallan en casos distintos. El análisis de sensibilidad mostró que la red se apoya en una correlación de las primeras generaciones (ataque especial alto y rareza) que no se generaliza del todo.
5. Un modelo no puede superar la información de sus características. Los legendarios con estadísticas bajas solo se podrían reconocer con otras variables, como la línea evolutiva o la tasa de captura, no con una red más grande.

5. Ejercicio 4 (punto libre). De la convolución a la red convolucional

5.1 Planteamiento

La sección 5.8 del capítulo introduce la convolución y el max pooling. El punto libre se dedicó a: implementar las dos operaciones desde cero y verificarlas contra el capítulo, reproducir la figura 5.24, entrenar una red convolucional (CNN) con Fashion MNIST y compararla con la red del Ejercicio 2, y examinar lo que la CNN aprende.

5.2 Convolución y pooling desde cero

Se programó la convolución de dos formas: con ciclos, siguiendo literalmente la descripción del capítulo (dividir la imagen en submatrices de 3×3 y multiplicar cada una por el filtro), y vectorizada, para imágenes grandes. Ambas reproducen exactamente la matriz 4×4 del capítulo y coinciden con `scipy.signal.correlate2d` y con `tf.nn.conv2d`. Se aclaró además un detalle de nomenclatura: la operación que el capítulo y las librerías de redes llaman convolución es, en rigor, una correlación cruzada; la convolución matemática invierte el filtro y, con el filtro del capítulo, produce la misma matriz con el signo cambiado.

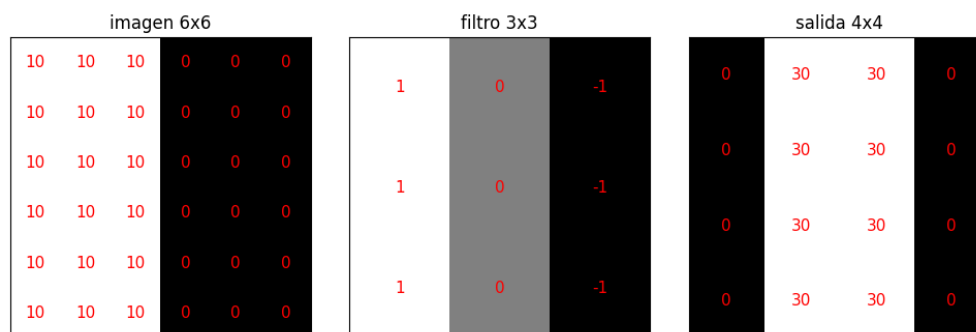


Figura 10. El filtro vertical del capítulo aplicado a una imagen dividida en dos mitades: la salida vale 30 justo en el borde y 0 en las zonas uniformes.

El max pooling, implementado con un cambio de forma del arreglo, reproduce la matriz de la figura 5.23, `[[192, 144], [255, 168]]`, igual que `MaxPooling2D` de Keras. Con la imagen *ascent* de SciPy (512×512), un filtro vertical y un max pooling de 2×2 se reprodujo la figura 5.24 del capítulo, que proviene del libro de Moroney citado en la bibliografía.

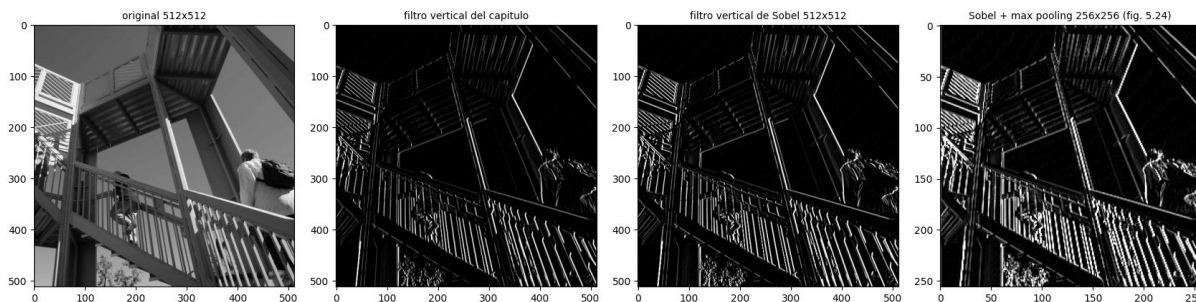


Figura 11. Reproducción de la figura 5.24: imagen original, filtro vertical del capítulo, filtro de Sobel y resultado tras el max pooling (256×256).

5.3 Red convolucional

La CNN tiene dos bloques de convolución y pooling (32 y 64 filtros de 3×3), seguidos de una capa densa de 128 neuronas con *Dropout* y la salida softmax. Tiene 225 034 parámetros, verificados a mano en el notebook. Su primera capa convolucional tiene solo 320 parámetros, frente a los 100 480 de la primera capa densa del Ejercicio 2, gracias al peso compartido: el mismo filtro de 9 pesos se aplica en todas las posiciones de la imagen. Para una comparación justa, la CNN y la red densa regularizada del Ejercicio 2 se entrenaron en el mismo notebook, con los mismos datos, optimizador, tamaño de lote y criterio de parada.

Tabla 12. Red densa frente a red convolucional en el conjunto de prueba de Fashion MNIST.

Modelo	Parámetros	Épocas	Tiempo (CPU)	Pérdida	Exactitud
Red densa (Ejercicio 2)	101 770	13	33 s	0.339	88.0 %
CNN	225 034	13	202 s	0.265	91.1 %

La CNN reduce el error de prueba de 12.0 % a 8.9 %, es decir, elimina una cuarta parte de los errores de la red densa. La mejora no es uniforme: la ganancia mayor, con diferencia, es la de buzo (+13.2 puntos, de 76.8 % a 90.0 %), seguida de camiseta (+5.3), sandalia (+3.3) y abrigo (+3.2), prendas que se distinguen por detalles locales. La clase *Camisa* sigue siendo con mucho la más difícil (72.9 %) y solo mejora 1.5 puntos, lo que indica que buena parte de su error viene de los propios datos (imágenes ambiguas) y no de la arquitectura.

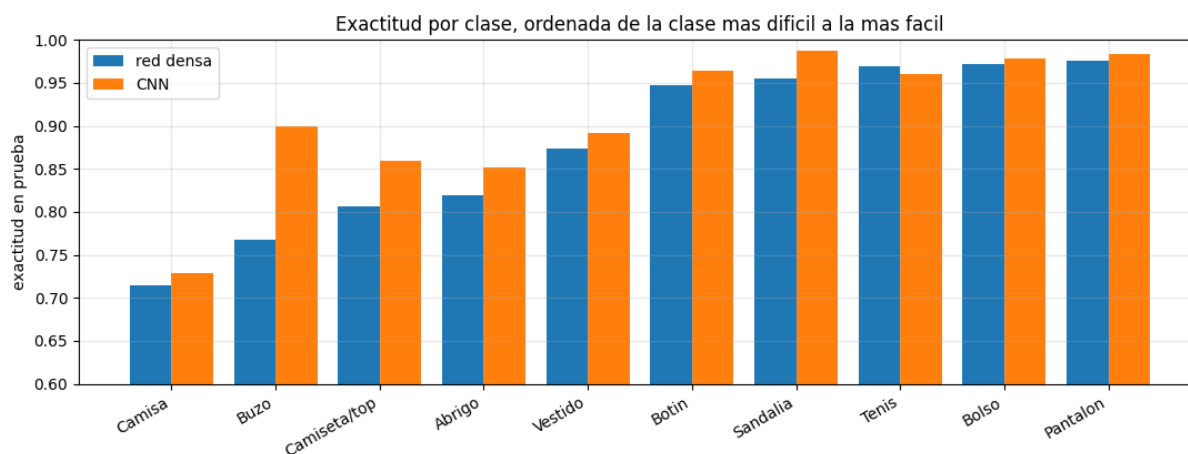


Figura 12. Exactitud por clase de la red densa y de la CNN, ordenada de la clase más difícil a la más fácil.

5.4 Dentro de la red entrenada

Los 32 filtros de la primera capa partieron de valores aleatorios, pero varios terminaron con la estructura de un detector de bordes: uno tiene correlación de +0.96 con el filtro vertical del capítulo y otro, de -0.97 con el filtro horizontal (el mismo patrón con signo invertido). Es lo que ilustra la figura 5.21 del capítulo: las primeras capas aprenden rasgos simples y genéricos. Los mapas de características muestran que cada filtro resalta un aspecto distinto de la misma prenda.

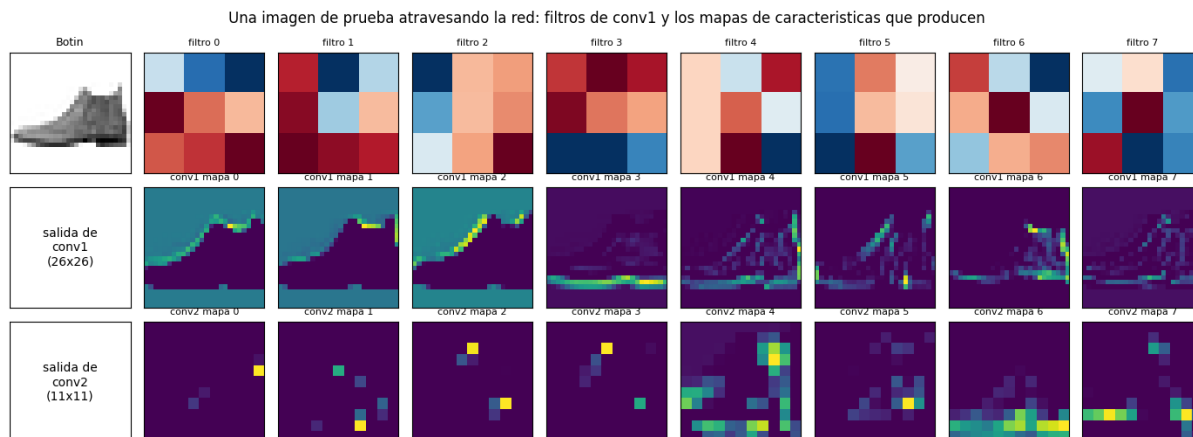


Figura 13. Una imagen de prueba atravesando la red: filtros de la primera capa y mapas de características de las dos capas convolucionales.

Como verificación final, se aplicaron los 32 filtros aprendidos a una imagen con la función de convolución escrita en NumPy, se sumó el sesgo y se aplicó ReLU. El resultado coincide con la salida de la capa Conv2D con una diferencia máxima de 9×10^{-8} . La capa convolucional de Keras es exactamente la operación del capítulo.

5.5 Conclusiones

1. Las implementaciones propias reproducen todos los ejemplos numéricos de la sección 5.8 y la figura 5.24, y coinciden con SciPy y TensorFlow.
2. La CNN alcanza 91.1 % de exactitud frente a 88.0 % de la red densa, con una primera capa de solo 320 parámetros gracias al peso compartido.
3. Los filtros aprendidos se parecen a los detectores de bordes diseñados a mano, lo que ilustra que con *Deep Learning* los rasgos se aprenden implícitamente.
4. La clase *Camisa* casi no mejora con la CNN, lo que señala un límite de los datos y no del modelo.

6. Observaciones sobre el material del capítulo

Durante las verificaciones se encontraron algunos detalles del material del curso que se reportan con el ánimo de contribuir a su mejora:

- **Derivada de la sigmoide en el programa del Ejemplo 1 (sección 5.6).** La función dSig calcula $\text{sig}(s.T[0]) * (\text{sig}(s.T[0]))$, es decir σ^2 , en lugar de $\sigma(1 - \sigma)$, que es la expresión establecida en la sección 5.2.2. Con la corrección, el Ejemplo 1 reduce su error final 110 veces y la primera salida alcanza su rótulo de 0.01. En redes con capas ocultas que deben aprender una representación nueva, como la XOR, la diferencia es decisiva (0 de 40 entrenamientos exitosos frente a 39 de 40).
- **Primera submatriz de la convolución (sección 5.8.1).** La submatriz que se muestra para el primer elemento tiene un 1 en la esquina inferior derecha, pero el valor de la imagen en esa posición es 2, que es el que se usa en el cálculo; el resultado, -5 , es correcto.
- **Tamaño de la imagen en la convolución.** El texto dice que "la matriz de la imagen 4 X 4 se divide en 16 matrices 3 X 3"; la imagen es de 6×6 y la matriz resultante es la de 4×4 .

7. Uso de herramientas de IA Generativa

Los parámetros de entrega del curso establecen que, cuando se utilicen herramientas de IA Generativa, se deben hacer comentarios sobre qué herramientas se emplearon, los prompts utilizados y hasta qué punto sirvieron para el desarrollo de la documentación, los ejercicios o los proyectos. Este capítulo atiende esa exigencia.

7.1 Herramientas empleadas

Tabla 13. Herramientas empleadas en el desarrollo del capítulo.

Herramienta	Uso que se le dio	Utilidad observada
Chat gpt	Propuesta y escritura de notebooks, verificación de resultados.	Requirió revisar sus afirmaciones contra los resultados, porque cometió errores de interpretación (sección 7.3).
Documentación oficial de TensorFlow/Keras, scikit-learn, SciPy y kagglehub	Consulta de la interfaz de las funciones utilizadas.	Necesaria para confirmar el comportamiento de funciones como <code>class_weight</code> , <code>EarlyStopping</code> y <code>kagglehub.dataset_download</code> .
PokémonDB y Bulbapedia	Fuente de las estadísticas oficiales de los 23 ejemplos de internet y de las listas oficiales de legendarios y míticos.	Indispensable para el Ejercicio 3; los datos se contrastaron entre ambas fuentes.

7.2 Prompts utilizados

El trabajo se hizo en una conversación con Chat GPT:

- Explicame como puedo descargar y conocer los datos de Kaggle utilizando Google Colab
- Se presenta un error en el código Explicame que sucede y como se puede corregir
- Como se puede verificar los resultados obtenidos en la ejecución del siguiente programa.
- Como utilizar TensorFlow en Google Colab.

7.3 Alcance y limitaciones observadas

La herramienta escribió y ejecutó el código de los cuatro notebooks y redactó sus explicaciones y este documento. También propuso las verificaciones descritas en la sección 1.1. Dos hallazgos surgieron de esas verificaciones: el error en la derivada de la sigmoide del programa de clase y los rótulos inconsistentes del conjunto de Kaggle.

Su principal limitación fue que, en varias ocasiones, redactó interpretaciones antes de tener los resultados, y algunas resultaron falsas al contrastarlas con los datos. Se corrigieron, y es importante reportarlas porque muestran por qué no se puede aceptar un texto generado sin verificarlo:

- Atribuyó a Calyrex una estadística equivocada (Total de 520 en lugar de 500); se detectó al comparar con la fuente.

- Propuso que la red del Ejercicio 3 distinguía a los legendarios por tener estadísticas "equilibradas". Al medirlo, la hipótesis no se sostuvo (la mediana del indicador era igual en ambas clases), y se reemplazó por el análisis de sensibilidad de la sección 4.6, que sí explica el comportamiento de la red.
- Anticipó que la CNN corregiría sobre todo los errores en la clase *Camisa* y que eliminaría un tercio de los errores. Los resultados mostraron que Camisa casi no mejora y que la reducción es de un cuarto; el texto se reescribió con las cifras reales.
- Describió la frontera de decisión de la XOR al revés (qué región correspondía a cada clase); se corrigió al revisar la figura.

Otra limitación práctica fue que el entorno de ejecución de la herramienta no tenía acceso a la API de Kaggle, así que el notebook del Ejercicio 3 se ejecutó con la copia de respaldo del conjunto, tal como se indica en su salida. En Colab la descarga con kagglehub funciona directamente.

8. Programas, repositorio y ejecución

8.1 Estructura de los archivos

```
5. Redes neuronales/
├── README.md
├── requirements.txt
├── Documento_Reses_Neuronales.pdf
├── Documento_Reses_Neuronales.docx
├── notebooks/
│   ├── Ejercicio_1_Compuertas_NAND_XOR.ipynb
│   ├── Ejercicio_2_Fashion_MNIST.ipynb
│   ├── Ejercicio_3_Kaggle_Pokemon_Legendarios.ipynb
│   └── Ejercicio_4_Convolucion_Pooling_CNN.ipynb
└── img/      figuras generadas por los notebooks
```

8.2 Ejecución en Google COLAB

Cada notebook se puede abrir directamente en Colab con los siguientes enlaces. Todas las librerías necesarias vienen preinstaladas; para los ejercicios 2 y 4 se recomienda activar la GPU.

- [Ejercicio 1. Compuertas NAND y XOR](#)
- [Ejercicio 2. Fashion MNIST](#)
- [Ejercicio 3. Dataset de Kaggle](#)
- [Ejercicio 4. Convolución, pooling y CNN](#)

8.3 Ejecución local con Jupyter

```
pip install -r requirements.txt
jupyter notebook notebooks/
```

Tiempos aproximados en CPU: Ejercicio 1, 6 minutos (la mayor parte es el experimento de robustez); Ejercicio 2, 6 minutos; Ejercicio 3, 5 minutos; Ejercicio 4, 7 minutos. Todos los notebooks fijan semillas aleatorias, de modo que en CPU los resultados se repiten exactamente; en una GPU las cifras de los ejercicios con TensorFlow pueden variar en décimas de punto porcentual.

9. Bibliografía

- [1]Martínez, J. J. (2026). *Introducción a las redes neuronales artificiales*, capítulo 5. Material del curso Inteligencia Artificial y Minirobots, Universidad Nacional de Colombia.
- [2]Moroney, L. (2020). *AI and Machine Learning for Coders: A Programmer's Guide to Artificial Intelligence*. O'Reilly.
- [3]Ng, A. *Machine Learning*. Coursera.
- [4]He, K., Zhang, X., Ren, S. y Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of CVPR*.
- [5]Minsky, M. y Papert, S. (1969). *Perceptrons*. MIT Press.
- [6]Rumelhart, D., Hinton, G. y Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533-536.
- [7]Xiao, H., Rasul, K. y Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747.
- [8]Barradas, A. *Pokemon with stats* [conjunto de datos]. Kaggle, identificador abcsds/pokemon.
- [9]PokémonDB, <https://pokemondb.net/pokedex>, y Bulbapedia, <https://bulbapedia.bulbagarden.net>. Consultados en septiembre de 2026.